Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота №5

із дисципліни «*Технології розроблення програмного забезпечення*»

Тема: «Патерни проектування»

Виконала:
Студент групи ІА-31
Самелюк А.С.

Перевірив:
Мягкий М.Ю.

Київ-2025

Тема: Патерни проектування.

Тема проєкту: Онлайн-щоденник (strategy, adapter, observer, facade, composite, client-server). Веб-застосунок дозволяє користувачу створювати, редагувати та організовувати особисті записи з можливістю пошуку, сортування та додавання тегів, а також отримання email-нагадувань.

Мета: Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи.

Посилання на репозиторій з проєктом та звітами:
https://github.com/Sameliuk/OnlineDiary_trpz/tree/main

https://github.com/Sameliuk/OnlineDiary_trpz/tree/reports

Хід роботи

1. Ознайомитись з короткими теоретичними відомостями.
2. Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

   В даній частині було реалізовану логіку з нотатками та відредаговано логіку з нагадуваннями. Було реалізовано відповідні сервіси, контролери та інтерфейси для виконання певного функціоналу.

Details.cshtml | IEmailSender.cs | EmailService.cs | EmailServiceAdapter.cs | Remin

D:\KPI\OnlineDiaryApp\OnlineDiaryApp\OnlineDiaryApp.csproj

```
@model OnlineDiaryApp.Models.Note

<link rel="stylesheet" href="~/css/notes.css" />

<div class="note-details">
    <h2>@Model.Title</h2>
    <p class="note-date">Створено: @Model.CreatedAt.ToString("f")</p>

    <div class="note-content">
        <h4> Зміст</h4>
        <p>@Model.Content</p>
    </div>

    <div class="note-tags">
        <h4> Теги:</h4>
        @if (Model.Tags.Any())
        {
            @foreach (var tag in Model.Tags)
            {
                <span class="note-tag">@tag.Name</span>
            }
        }
        else
        {
            <span class="note-tag-empty">Без тегів</span>
        }
    </div>

    @if (ViewBag.Reminder != null)
    {
        <div class="note-reminder">
            <h4> Нагадування:</h4>
            <p>@(((DateTime)ViewBag.Reminder.RemindAt).ToString("f"))</p>
        </div>
    }

    <div class="note-actions">
        <a asp-action="Edit" asp-route-id="@Model.Id" class="btn-edit">Редагувати</a>
        <a asp-action="Delete" asp-route-id="@Model.Id" class="btn-delete">Видалити</a>
        <a asp-action="Index" class="btn-back">← Назад</a>
    </div>
</div>
```

Код 1 – Details (Views.Note)

NoteController.cs | Details.cshtml | IEmailSender.cs | EmailService.cs | EmailServiceAdapter.cs

OnlineDiaryApp | OnlineDiaryApp.Controllers.NoteController

```
1   using Microsoft.AspNetCore.Mvc;
2   using OnlineDiaryApp.Models;
3   using OnlineDiaryApp.Services;
4
5   namespace OnlineDiaryApp.Controllers
6   {
        1 reference
7       public class NoteController : Controller
8       {
9           private readonly NoteService _noteService;
10          private readonly ReminderService _reminderService;
11          private readonly TagService _tagService;
12
            0 references
13          public NoteController(NoteService noteService, ReminderService reminderService, TagService tagService)
14          {
15              _noteService = noteService;
16              _reminderService = reminderService;
17              _tagService = tagService;
18          }
19
            0 references
20          public async Task<IActionResult> Index(string? sortBy, string? tag)
21          {
22
23              var userIdString = HttpContext.Session.GetString("UserId");
24              if (!int.TryParse(userIdString, out int userId))
25                  return RedirectToAction("Login", "User");
26
27              ISortStrategy? strategy = null;
28              if (!string.IsNullOrEmpty(sortBy))
29              {
30                  strategy = sortBy.ToLower() switch
31                  {
32                      "date" => new SortByDateStrategy(),
33                      "tag" when !string.IsNullOrEmpty(tag) => new SortByTagStrategy(tag),
34                      "title" => new SortByTitleStrategy(),
35                      _ => null
36                  };
37              }
38
39              var notes = await _noteService.GetAllNotesByUserAsync(userId, strategy);
40
41              ViewBag.SortBy = sortBy;
42              ViewBag.SelectedTag = tag;
43              ViewBag.Tags = await _tagService.GetAllTagsAsync(userId);
44
45              return View(notes);
46          }
47
            0 references
48          public async Task<IActionResult> Create()
49          {
50              var userIdString = HttpContext.Session.GetString("UserId");
51              if (!int.TryParse(userIdString, out int userId))
52                  return RedirectToAction("Login", "User");
53
54              var tags = await _tagService.GetAllTagsAsync(userId);
55              ViewBag.Tags = tags ?? new List<Tag>();
56
57              return View();
58          }
59
60          [HttpPost]
            0 references
61          public async Task<IActionResult> Create(string title, string content, List<int>? tagIds, DateTime? reminderDate)
62          {
63              var userIdString = HttpContext.Session.GetString("UserId");
64              if (!int.TryParse(userIdString, out int userId))
65                  return RedirectToAction("Index");
66
67              var note = await _noteService.CreateNoteAsync(title, content, userId, tagIds ?? new List<int>());
68
69              if (reminderDate.HasValue)
70              {
71                  await _reminderService.CreateReminderAsync(note.Id, reminderDate.Value, userId);
72              }
73
74              return RedirectToAction("Index");
75          }
76
            0 references
77          public async Task<IActionResult> Edit(int id)
78          {
79              var note = await _noteService.GetNoteByIdAsync(id);
80              if (note == null)
81                  return NotFound();
82
83              var userIdString = HttpContext.Session.GetString("UserId");
84              if (!int.TryParse(userIdString, out int userId))
85                  return RedirectToAction("Login", "User");
86
87              ViewBag.Tags = await _tagService.GetAllTagsAsync(userId) ?? new List<Tag>();
88
89              var reminder = await _reminderService.GetReminderByNoteIdAsync(id);
90              ViewBag.Reminder = reminder;
91
92              return View(note);
93          }
94
```

Код 2.1 – NoteController (Controllers)

```
95          [HttpPost]
            0 references
96          public async Task<IActionResult> Edit(int id, string title, string content, List<int>? tagIds, DateTime? reminderDate)
97          {
98              var note = await _noteService.GetNoteByIdAsync(id);
99              if (note == null)
100                 return NotFound();
101
102             note.Title = title;
103             note.Content = content;
104
105             await _noteService.UpdateNoteAsync(note, tagIds ?? new List<int>());
106
107             var existingReminder = await _reminderService.GetReminderByNoteIdAsync(note.Id);
108             if (reminderDate.HasValue)
109             {
110                 if (existingReminder != null)
111                 {
112                     await _reminderService.UpdateReminderAsync(existingReminder, reminderDate.Value);
113                 }
114                 else
115                 {
116                     await _reminderService.CreateReminderAsync(note.Id, reminderDate.Value, note.UserId);
117                 }
118             }
119
120             return RedirectToAction("Index");
121         }
122
            0 references
123         public async Task<IActionResult> Delete(int id)
124         {
125             await _noteService.DeleteNoteAsync(id);
126             return RedirectToAction("Index");
127         }
128
            0 references
129         public async Task<IActionResult> Search(string keyword)
130         {
131             var userIdString = HttpContext.Session.GetString("UserId");
132             if (!int.TryParse(userIdString, out int userId))
133                 return View(new List<Note>());
134
135             var notes = await _noteService.SearchByTitleAsync(keyword);
136             notes = notes.Where(n => n.UserId == userId);
137             return View("Index", notes);
138         }
139
            0 references
140         public async Task<IActionResult> Details(int id)
141         {
142             var note = await _noteService.GetNoteByIdAsync(id);
143             if (note == null)
144                 return NotFound();
145
146             var reminder = await _reminderService.GetReminderByNoteIdAsync(id);
147             ViewBag.Reminder = reminder;
148
149             return View(note);
150         }
151
152     }
153 }
154
```

Код 2.2 – NoteController (Controllers)

```
ReminderBack...ngService.cs*   NoteController.cs   Details.cshtml   IEmailSender.cs
OnlineDiaryApp                                      ReminderBackgroundService

using OnlineDiaryApp.Services;

2 references
public class ReminderBackgroundService : BackgroundService
{
    private readonly IServiceProvider _serviceProvider;

    0 references
    public ReminderBackgroundService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    0 references
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            try
            {
                using var scope = _serviceProvider.CreateScope();
                var reminderService = scope.ServiceProvider.GetRequiredService<ReminderService>();

                var now = DateTime.UtcNow;

                var reminders = (await reminderService.GetAllRemindersAsync())
                                .Where(r => r.Status == "active" && r.RemindAt <= now)
                                .ToList();

                foreach (var reminder in reminders)
                {
                    await reminderService.SendReminderEmailAsync(reminder);
                    await reminderService.UpdateReminderAsync(reminder, newStatus: "sent");
                }
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Reminder service error: {ex.Message}");
            }

            await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
        }
    }
}
```

Код 3 – ReminderBackgroundService (Services)

3. Реалізувати один з розглянутих шаблонів за обраною темою.

Для даного проєкту було обрано патерн Adapter (Адаптер). Шаблон реалізований таким чином:

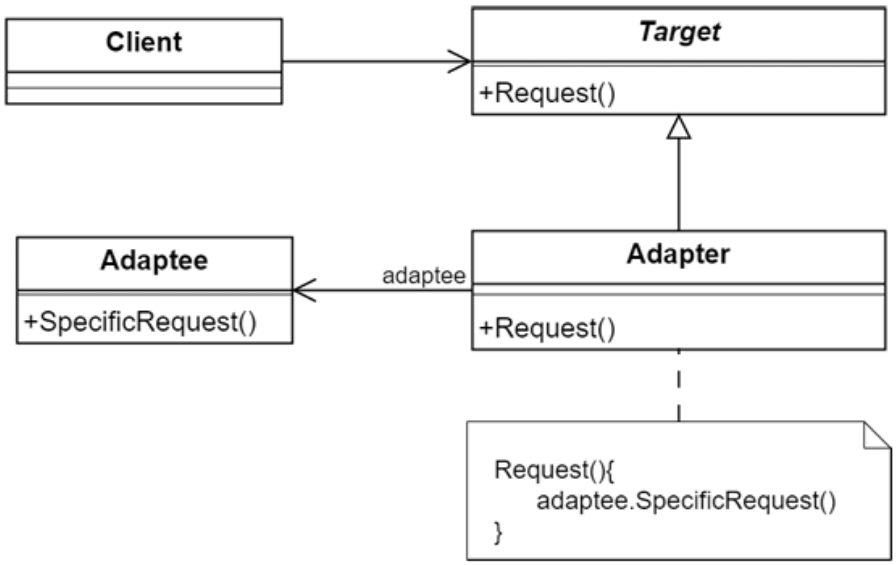


Рис.1 – Структура патерну Адаптер

Target: IEmailSender – це інтерфейс, який описує метод SendEmailAsync. Він задає єдиний контракт для всіх способів відправлення електронних листів (SMTP, API, тестовий варіант тощо).

Client: ReminderService – це клієнт, який використовує IEmailSender, не знаючи, як саме реалізовано відправку листів. Він просто викликає SendEmailAsync, а вже адаптер вирішує, який саме механізм відправлення використати.

Adaptee: EmailService – це клас-адаптований об'єкт, який уже вміє відправляти листи через SMTP. Однак його інтерфейс не відповідає тому, що очікує система (IEmailSender).

Adapter: EmailServiceAdapter – це адаптер, який "обгортає" EmailService і перетворює його інтерфейс у формат IEmailSender. Завдяки йому EmailService можна легко використовувати в коді, що працює з інтерфейсом IEmailSender.

В результаті: можна легко змінювати або підключати різні способи відправлення електронних листів (через SMTP, API, інший сервіс) без зміни коду клієнта — ReminderService. Сервіс працює лише з інтерфейсом IEmailSender, не знаючи, як саме реалізовано відправку. Це і є суть патерну Адаптер – узгодження несумісних інтерфейсів за допомогою проміжного класу, що перетворює виклики клієнта у форму, яку розуміє адаптований об'єкт (EmailService).

4. Реалізувати не менше 3-х класів відповідно до обраної теми.

ReminderService.cs* | Index.cshtml | tag.css | Create.cshtml | note.c

OnlineDiaryApp | OnlineDiaryApp.Servic

```
using OnlineDiaryApp.Models;
using OnlineDiaryApp.Interfaces;
using OnlineDiaryApp.Repositories.Interfaces;

namespace OnlineDiaryApp.Services
{
    7 references
    public class ReminderService
    {
        private readonly IReminderRepository _reminderRepository;
        private readonly IUserRepository _userRepository;
        private readonly IEmailSender _emailSender;

        0 references
        public ReminderService(
            IReminderRepository reminderRepository,
            IUserRepository userRepository,
            IEmailSender emailSender)
        {
            _reminderRepository = reminderRepository;
            _userRepository = userRepository;
            _emailSender = emailSender;
        }

        3 references
        public async Task CreateReminderAsync(int noteId, DateTime remindAt, int userId)
        {
            var utcRemindAt = DateTime.SpecifyKind(remindAt, DateTimeKind.Utc);

            var reminder = new Reminder
            {
                NoteId = noteId,
                UserId = userId,
                RemindAt = utcRemindAt,
                Status = "active"
            };

            await _reminderRepository.AddAsync(reminder);
            await _reminderRepository.SaveChangesAsync();

            await SendReminderEmailAsync(reminder);
        }

        2 references
        public async Task SendReminderEmailAsync(Reminder reminder)
        {
            var user = reminder.User ?? await _userRepository.GetByIdAsync(reminder.UserId);
            var note = reminder.Note;

            if (user != null && note != null && !string.IsNullOrEmpty(user.Email))
            {
                await _emailSender.SendEmailAsync(
                    user.Email,
                    $"Нагадування: {note.Title}",
                    $"Нагадування по нотатці:\n\n{note.Content}"
                );
            }
        }

        1 reference
        public async Task<IEnumerable<Reminder>> GetAllRemindersAsync()
        {
            return await _reminderRepository.GetAllAsync();
        }

        2 references
        public async Task UpdateReminderAsync(Reminder reminder, DateTime? newRemindAt = null, string? newStatus = null)
        {
            if (newRemindAt.HasValue)
                reminder.RemindAt = DateTime.SpecifyKind(newRemindAt.Value, DateTimeKind.Utc);

            if (!string.IsNullOrEmpty(newStatus))
                reminder.Status = newStatus;

            await _reminderRepository.UpdateAsync(reminder);
            await _reminderRepository.SaveChangesAsync();
        }

        0 references
        public async Task DeleteReminderAsync(int reminderId)
        {
            var reminder = await _reminderRepository.GetByIdAsync(reminderId);
            if (reminder != null)
            {
                await _reminderRepository.DeleteAsync(reminder.Id);
                await _reminderRepository.SaveChangesAsync();
            }
        }

        3 references
        public async Task<Reminder?> GetReminderByNoteIdAsync(int noteId)
        {
            return await _reminderRepository.GetByNoteIdAsync(noteId);
        }

    }
}
```

Код 4 – ReminderService (Services)

```
EmailServiceAdapter.cs   ReminderBack...ungService.cs*   ReminderService.cs*   note.css
OnlineDiaryApp                                            OnlineDiaryApp.Services.EmailServic
1   using OnlineDiaryApp.Interfaces;
2
3   namespace OnlineDiaryApp.Services
4   {
        2 references
5       public class EmailServiceAdapter : IEmailSender
6       {
7           private readonly EmailService _emailService;
8
            0 references
9           public EmailServiceAdapter(EmailService emailService)
10          {
11              _emailService = emailService;
12          }
13
            2 references
14          public async Task SendEmailAsync(string to, string subject, string body)
15          {
16              await _emailService.SendEmailAsync(to, subject, body);
17          }
18      }
19  }
20
```

Код 5 – EmailServiceAdapter (Services)

```
EmailService.cs*   EmailServiceAdapter.cs   ReminderBack...ungService.cs*   ReminderService.cs*   Edit.cshtml
OnlineDiaryApp                                            OnlineDiaryApp.Services.EmailService
1   using MailKit.Net.Smtp;
2   using MimeKit;
3
4   namespace OnlineDiaryApp.Services
5   {
        3 references
6       public class EmailService
7       {
8           private readonly string _smtpServer = "smtp.gmail.com";
9           private readonly int _port = 587;
10          private readonly string _username = "";
11          private readonly string _password = "";
12
            1 reference
13          public async Task SendEmailAsync(string to, string subject, string body)
14          {
15              var message = new MimeMessage();
16              message.From.Add(new MailboxAddress("Online Diary", _username));
17              message.To.Add(MailboxAddress.Parse(to));
18              message.Subject = subject;
19              message.Body = new TextPart("plain") { Text = body };
20
21              using var client = new SmtpClient();
22              await client.ConnectAsync(_smtpServer, _port, MailKit.Security.SecureSocketOptions.StartTls);
23              await client.AuthenticateAsync(_username, _password);
24              await client.SendAsync(message);
25              await client.DisconnectAsync(true);
26          }
27      }
28  }
29
```

Код 6 – EmailService (Services)

```
IEmailSender.cs   EmailService.cs   EmailServiceAdapter.cs   ReminderBack.
OnlineDiaryApp                                              OnlineDiaryApp.Inte
1  namespace OnlineDiaryApp.Interfaces
2  {
       4 references
3      public interface IEmailSender
4      {
           2 references
5          Task SendEmailAsync(string to, string subject, string body);
6      }
7  }
8
```

Код 7 – IEmailSender (Interfaces)

5. Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

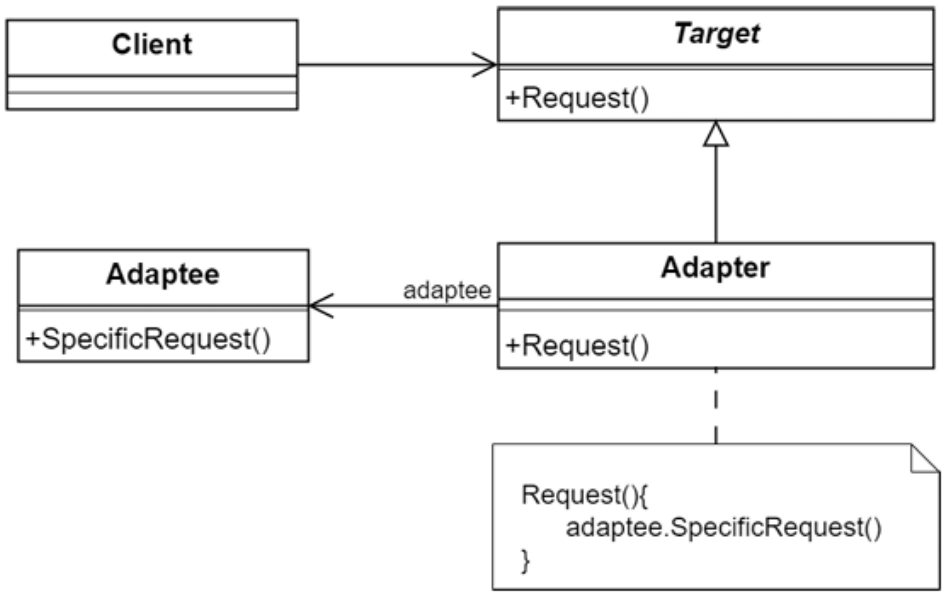
Висновки: Виконання лабораторної роботи дозволило ознайомитися з основними методами об'єктно-орієнтованого моделювання системи за допомогою UML. Було створено діаграму варіантів використання, яка наочно відображає взаємодію користувачів із системою та ключові функції, що вона надає. За допомогою діаграми класів було показано основні класи системи, їх атрибути та методи, а також зв'язки між класами, що дозволяє зрозуміти структурну організацію програми та логіку взаємодії об'єктів.

Відповіді на контрольні питання:

1. Яке призначення шаблону «Адаптер»?

Забезпечує сумісність між двома інтерфейсами, які спочатку не можуть працювати разом. Адаптер «обгортає» існуючий клас і перетворює його інтерфейс у той, який очікує клієнт.

2. Нарисуйте структуру шаблону «Адаптер».



3. Які класи входять в шаблон «Адаптер», та яка між ними взаємодія?

- Client — використовує об'єкт через інтерфейс Target.
- Target — описує інтерфейс, який очікує клієнт.
- Adaptee — існуючий клас із несумісним інтерфейсом.
- Adapter — реалізує Target, але всередині викликає методи Adaptee.

4. Яка різниця між реалізацією «Адаптера» на рівні об'єктів та на рівні класів?

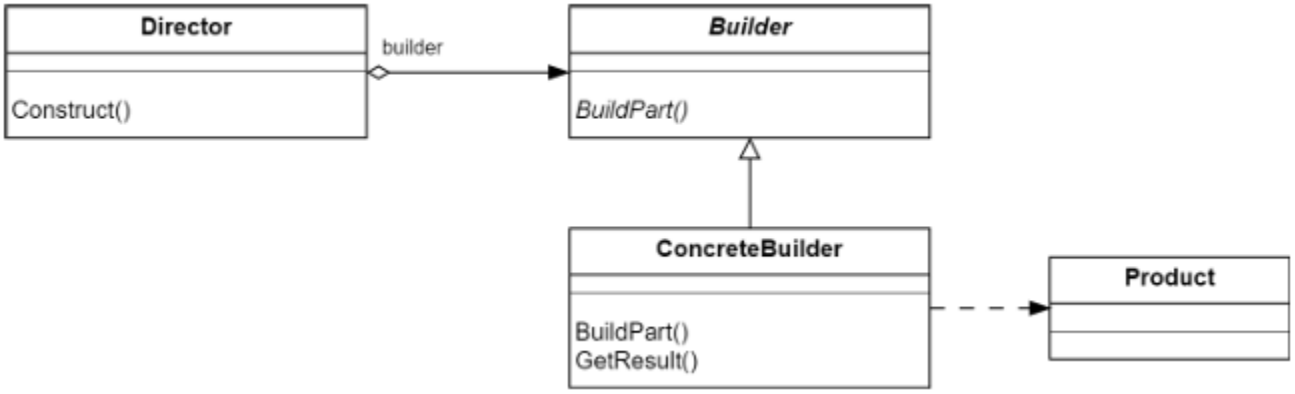
- Об'єктний адаптер — використовує композицію (всередині містить екземпляр Adaptee).
- Класовий адаптер — використовує спадкування (наслідує Adaptee і реалізує Target).

  Об'єктний підхід гнучкіший, бо дозволяє адаптувати кілька класів одночасно.

5. Яке призначення шаблону «Будівельник»?

Відокремлює процес створення складного об'єкта від його представлення, дозволяючи будувати різні варіанти об'єкта з однієї і тієї ж послідовності кроків.

6. Нарисуйте структуру шаблону «Будівельник».



7. Які класи входять в шаблон «Будівельник», та яка між ними взаємодія?

- Builder — інтерфейс, що описує кроки побудови об’єкта.
- ConcreteBuilder — конкретна реалізація, що створює частини продукту.
- Director — керує послідовністю викликів методів будівельника.
- Product — кінцевий складний об’єкт.

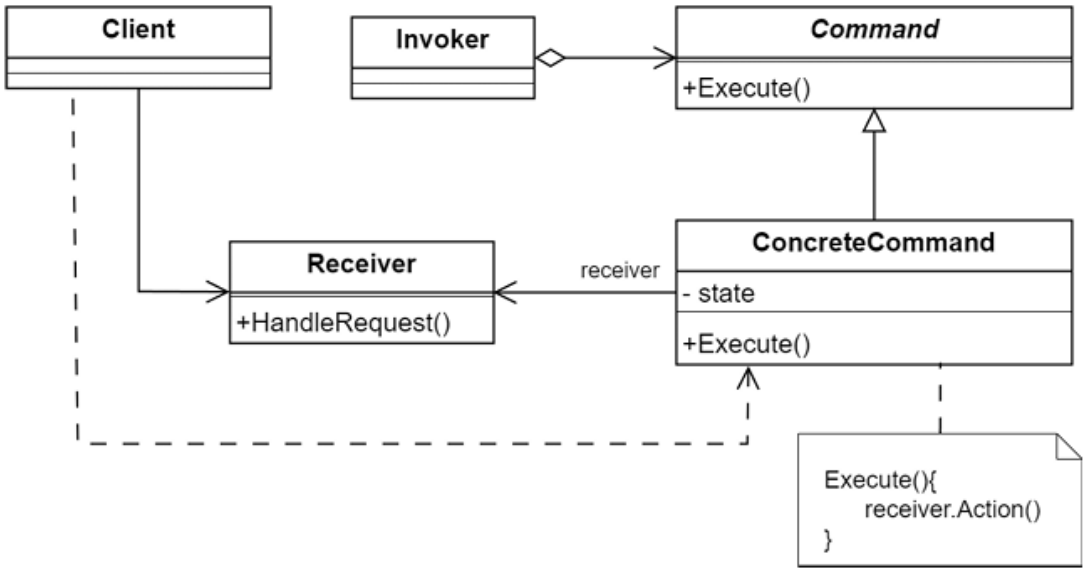
8. У яких випадках варто застосовувати шаблон «Будівельник»?

Коли об’єкт потрібно створювати поступово або в кілька кроків, і є багато варіантів конфігурації цього об’єкта.

9. Яке призначення шаблону «Команда»?

Інкапсулює запит (дію) як об’єкт, що дозволяє параметризувати об’єкти діями, ставити їх у чергу або скасовувати операції.

10. Нарисуйте структуру шаблону «Команда».



11. Які класи входять в шаблон «Команда», та яка між ними взаємодія?

- Command — інтерфейс із методом Execute().
- ConcreteCommand — реалізує команду, викликаючи метод у Receiver.
- Receiver — виконує реальну роботу.
- Invoker — зберігає команду і викликає її виконання.
- Client — створює команду й призначає її виконавцю.

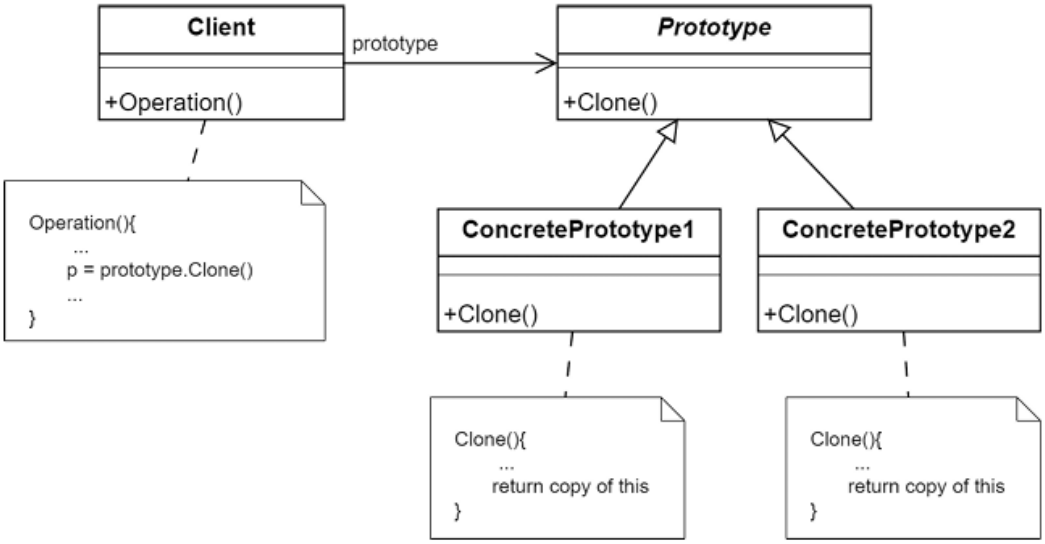
12. Розкажіть як працює шаблон «Команда».

Клієнт створює команду і задає отримувача. Коли Invoker викликає метод Execute(), команда звертається до Receiver, який виконує необхідну дію. Це дозволяє відділити відправника запиту від його виконання.

13. Яке призначення шаблону «Прототип»?

Створює нові об’єкти шляхом копіювання вже існуючих екземплярів (прототипів), замість створення через конструктор.

14. Нарисуйте структуру шаблону «Прототип».



15. Які класи входять в шаблон «Прототип», та яка між ними взаємодія?

- Prototype — оголошує метод Clone().
- ConcretePrototype — реалізує клонування самого себе.
- Client — створює нові об’єкти через метод Clone().

16. Які можна привести приклади використання шаблону «Ланцюжок відповідальності»?

- Обробка запитів у системах підтримки (спочатку оператор, потім менеджер, потім директор).
- Фільтрація запитів у веб-додатках (middleware).
- Обробка подій у GUI (коли подія передається вгору по ієрархії елементів інтерфейсу).